Problem statement:

Take any text file input.txt consisting only of alphabets a-z. Find frequencies of each alphabet. Compute the Huffman encoding.

- Use the coding scheme to encode the text file to encoded.txt.
- Use the coding scheme to decode the text file encode.txt to decoded.txt.
- Verify the correctness of decoded.txt by comparing with input.txt

Source code:

```
#include <stdio.h>
#include <stdlib.h>
#include <string.h>

#define MAX_SIZE 256
#define MAX_TREE_HT 256

// Structure for each node of the Huffman tree
typedef struct MinHeapNode
{
	char data;		// character
	unsigned freq;		// frequency of character
	struct MinHeapNode *left;  // left child
	struct MinHeapNode *right; // right child
} MinHeapNode;

// MinHeap used for building the Huffman tree
typedef struct MinHeap
{
	unsigned size;
	unsigned capacity;
	MinHeapNode **array;
} MinHeap;

// -------- Utility functions for Heap & Tree --------

// Create a new node with given character and frequency
MinHeapNode *newNode(char data, unsigned freq)
{
	MinHeapNode *temp = (MinHeapNode *)malloc(sizeof(MinHeapNode));
	temp->data = data;
	temp->freq = freq;
	temp->left = temp->right = NULL;
	return temp;
}

// Initialize a min heap with given capacity
MinHeap *createMinHeap(unsigned capacity)
{
	MinHeap *minHeap = (MinHeap *)malloc(sizeof(MinHeap));
	minHeap->size = 0;
	minHeap->capacity = capacity;
	minHeap->array = (MinHeapNode **)malloc(minHeap->capacity * sizeof(MinHeapNode *));
	return minHeap;
}
```

Do not read this document

```
// Swap two nodes inside the heap
void swapMinHeapNode(MinHeapNode **a, MinHeapNode **b)
{
	MinHeapNode *t = *a;
	*a = *b;
	*b = t;
}

// Heapify to maintain min-heap property
void minHeapify(MinHeap *minHeap, int idx)
{
	int smallest = idx;
	int left = 2 * idx + 1;
	int right = 2 * idx + 2;

	if (left < (int)minHeap->size &&
	   minHeap->array[left]->freq < minHeap->array[smallest]->freq)
		smallest = left;

	if (right < (int)minHeap->size &&
	   minHeap->array[right]->freq < minHeap->array[smallest]->freq)
		smallest = right;

	if (smallest != idx)
	{
		swapMinHeapNode(&minHeap->array[smallest], &minHeap->array[idx]);
		minHeapify(minHeap, smallest);
	}
}

// Extract the minimum frequency node from heap
MinHeapNode *extractMin(MinHeap *minHeap)
{
	MinHeapNode *temp = minHeap->array[0];
	minHeap->array[0] = minHeap->array[minHeap->size - 1];
	--minHeap->size;
	minHeapify(minHeap, 0);
	return temp;
}

// Insert a new node into the heap
void insertMinHeap(MinHeap *minHeap, MinHeapNode *node)
{
	++minHeap->size;
	int i = minHeap->size - 1;
	while (i && node->freq < minHeap->array[(i - 1) / 2]->freq)
	{
		minHeap->array[i] = minHeap->array[(i - 1) / 2];
		i = (i - 1) / 2;
	}
	minHeap->array[i] = node;
}

// Build the heap by arranging nodes properly
void buildMinHeap(MinHeap *minHeap)
{
	int n = minHeap->size - 1;
```

Do not read this document

```
	for (int i = (n - 1) / 2; i >= 0; i--)
		minHeapify(minHeap, i);
}

// Leaf node check (no children)
int isLeaf(MinHeapNode *root)
{
	return !(root->left) && !(root->right);
}

// Create and build min heap from given frequency table
MinHeap *buildAndCreateMinHeap(char data[], int freq[], int size)
{
	MinHeap *minHeap = createMinHeap(size);
	for (int i = 0; i < size; ++i)
		minHeap->array[i] = newNode(data[i], freq[i]);
	minHeap->size = size;
	buildMinHeap(minHeap);
	return minHeap;
}

// -------- Building Huffman Tree --------
MinHeapNode *buildHuffmanTree(char data[], int freq[], int size)
{
	MinHeapNode *left, *right, *top;
	MinHeap *minHeap = buildAndCreateMinHeap(data, freq, size);

	// Keep extracting two smallest nodes and merge until one tree remains
	while (minHeap->size != 1)
	{
		left = extractMin(minHeap);
		right = extractMin(minHeap);

		top = newNode('$', left->freq + right->freq); // internal node
		top->left = left;
		top->right = right;

		insertMinHeap(minHeap, top);
	}
	return extractMin(minHeap);
}

// -------- Generating Huffman Codes --------
void storeCodes(MinHeapNode *root, char *str,
		char codes[MAX_SIZE][MAX_TREE_HT], int top)
{
	if (root->left)
	{
		str[top] = '0';
		storeCodes(root->left, str, codes, top + 1);
	}
	if (root->right)
	{
		str[top] = '1';
		storeCodes(root->right, str, codes, top + 1);
	}
	if (isLeaf(root))
```

```
        {
                str[top] = '\0';
                strcpy(codes[(unsigned char)root->data], str);
        }
}

// -------- Tree Serialization & Deserialization --------
// We write tree in preorder: '1' + char for leaf, '0' for internal
void writeTree(MinHeapNode *root, FILE *out)
{
        if (isLeaf(root))
        {
                fputc('1', out);
                fputc(root->data, out);
        }
        else
        {
                fputc('0', out);
                writeTree(root->left, out);
                writeTree(root->right, out);
        }
}

// Rebuild the tree while reading back the same format
MinHeapNode *readTree(FILE *in)
{
        int flag = fgetc(in);
        if (flag == EOF)
                return NULL;

        if (flag == '1')
        { // leaf
                char c = fgetc(in);
                return newNode(c, 0);
        }
        else if (flag == '0')
        { // internal
                MinHeapNode *node = newNode('$', 0);
                node->left = readTree(in);
                node->right = readTree(in);
                return node;
        }
        return NULL;
}

// -------- Encoding --------
void encodeFile(const char *inputFile, const char *encodedFile)
{
        FILE *in = fopen(inputFile, "r");
        if (!in)
        {
                printf("Input file not found!\n");
                return;
        }

        char text[10000];
        int freq[MAX_SIZE] = {0};
```

Do not read this document

```
    int length = 0;

    // Count frequency of each character
    char ch;
    while ((ch = fgetc(in)) != EOF)
    {
        text[length++] = ch;
        freq[(unsigned char)ch]++;
    }
    text[length] = '\0';
    fclose(in);

    // Collect non-zero frequency characters
    char data[MAX_SIZE];
    int f[MAX_SIZE], size = 0;
    for (int i = 0; i < MAX_SIZE; i++)
    {
        if (freq[i])
        {
            data[size] = (char)i;
            f[size] = freq[i];
            size++;
        }
    }

    MinHeapNode *root = buildHuffmanTree(data, f, size);

    // Generate codes
    char codes[MAX_SIZE][MAX_TREE_HT];
    char arr[MAX_TREE_HT];
    for (int i = 0; i < MAX_SIZE; i++)
        codes[i][0] = '\0';
    storeCodes(root, arr, codes, 0);

    // Write encoded file
    FILE *out = fopen(encodedFile, "w");
    fprintf(out, "HEADER\n");
    writeTree(root, out);
    fprintf(out, "\nDATA\n");
    for (int i = 0; i < length; i++)
        fprintf(out, "%s", codes[(unsigned char)text[i]]);
    fclose(out);

    printf("File encoded successfully -> %s\n", encodedFile);
}

// -------- Decoding --------
void decodeFile(const char *encodedFile, const char *decodedFile)
{
    FILE *in = fopen(encodedFile, "r");
    if (!in)
    {
        printf("Encoded file not found!\n");
        return;
    }

    char dummy[100];
```

Do not read this document

```
    fgets(dummy, sizeof(dummy), in); // Skip "HEADER\n"

    // Read Huffman tree from file
    MinHeapNode *root = readTree(in);

    // Move file pointer until DATA section
    while (fgets(dummy, sizeof(dummy), in))
    {
        if (strncmp(dummy, "DATA", 4) == 0)
            break;
    }

    // Read encoded binary string
    char encoded[10000];
    fscanf(in, "%s", encoded);
    fclose(in);

    // Decode using tree traversal
    FILE *out = fopen(decodedFile, "w");
    MinHeapNode *current = root;
    for (int i = 0; encoded[i] != '\0'; i++)
    {
        if (encoded[i] == '0')
            current = current->left;
        else
            current = current->right;

        if (isLeaf(current))
        {
            fputc(current->data, out);
            current = root;
        }
    }
    fclose(out);

    printf("File decoded successfully -> %s\n", decodedFile);
}

// -------- Menu System --------
void printMenu()
{
    printf("\n===== Huffman Coding Menu =====\n");
    printf("1. Encode a file\n");
    printf("2. Decode a file\n");
    printf("3. Exit\n");
    printf("Enter your choice: ");
}

int main()
{
    int choice;
    char inputFile[100], encodedFile[100], decodedFile[100];

    while (1)
    {
        printMenu();
        scanf("%d", &choice);
```

Do not read this document

```
        switch (choice)
        {
        case 1:
            printf("Enter input file name: ");
            scanf("%s", inputFile);
            printf("Enter encoded output file name: ");
            scanf("%s", encodedFile);
            encodeFile(inputFile, encodedFile);
            break;
        case 2:
            printf("Enter encoded file name: ");
            scanf("%s", encodedFile);
            printf("Enter decoded output file name: ");
            scanf("%s", decodedFile);
            decodeFile(encodedFile, decodedFile);
            break;
        case 3:
            exit(0);
        default:
            printf("Invalid choice!\n");
        }
    }
    return 0;
}
```

Output:

1) **Output 1:**
   i) Input.txt<<
   aaaabbbbccccdeeeeffffffgggghhiijjkkkllllmmmmmnoooooopqqrsssttttttuuuuuvvvvvwwwwwxxxxxyyyyzzzzz
   ii) Encode.txt<<
   HEADER
   00001a1e01v1m001u1x01w1f0001z1t01o01s1k00001i1j1y01b1l00001d1p01r1n01h1q01c1g
   DATA
   0000000000000000110101101011010110101111011110111101111011110000000100010001000101110111011101110111111111111111111111111111111110101110101100001100001100011100011011110111101111011110111101111011100110011001100110011111001110101010101010101010101110001111011111011111001010110101101011010011001100110011001100101000100010001000100010000100010010001000100100110011001100110011001100101010101010101010101110011100111001110011100100100010001000100010001000100010001000

2) **Output 2:**

i) Input.txt<<
tqmxypjoiuinyytunqqwejcbjztneozrutqzowpplvoienxchapcstmxoatquubxmlnzpbc
vzjlgishifyxvepsjywfjkvuxwnigd

ii) Encode.txt<<
HEADER
00001q1z01o1p0001b1s001k1a1l01u1j0001x1t01n1i0001m001d1r1g01v01h1f00
1c1e01w1y
DATA
1001000011000100011111001101110010101101101011101011111111110010 1
1010100000000011110111010111111000100001110001100110101110100100 00
1110010101101001000000010010111100011001101011110100010101111101 10
1010001110011011001010100111110001001100111000100000100101011001 00
0001100110010001000110000101110100001001101000111001101000010111 01
0111100111011010011101101011110111111111000110101110100110100101 11
1111111110110111011101010011010011010001111010101011110011111001 00
iii) Decode.txt<<
tqmxypjoiuinyytunqqwejcbjztneozrutqzowpplvoienxchapcstmxoatquubxmlnzpbc
vzjlgishifyxvepsjywfjkvuxwnigd

3) **Output 3:**

i) Input.txt<<
spejnnqfjuotsdskirbocuksrjajjqktikhyhdrkebthefyzdxulvowefmhzoafmqakqfeatpzz
xiezayimmgdw

ii) Encode.txt<<
HEADER
00001o1m01i1t0001w1x01n1r01f1z0001a1j01k1e0001y1u01h1q001s1d0001v1c
01l1g01p1b
DATA
1110011111010111001010100101011101101001110010000001111100111011 1
1100101000100101111111000011110011100110101110001011100110001001 1
0011101110100011001010101101011000110101110101011101010111111100 1
1110101011011011000011111101010011100111110101111000000000100010 110
1100001110100111000010000110000111011100010101101101101011100000 11
1111100111011101001001010110111100011000001000010001111101111101 01
000
iii) Decode.txt<<
spejnnqfjuotsdskirbocuksrjajjqktikhyhdrkebthefyzdxulvowefmhzoafmqakqfeatpzz
xiezayimmgdw